Phase A: Protocol Core — TDD Spec

> *Based on: 2026-04-12-agentxp-v4-design.md Model: opus (cryptography requires precise reasoning) All code, comments, error messages in English. Zero Chinese. Every task: write failing test → implement → green → commit*

A1: Type Definitions

Goal: Define all TypeScript types for the protocol layer. This is the foundation everything else builds on.

Directory: `packages/protocol/src/types.ts`

Tests to write first (`packages/protocol/tests/types.test.ts`):

```
// Test 1: SerendipEvent has required fields including version
const event: SerendipEvent = {
  v: 1,
  id: 'abc123',
  pubkey: 'deadbeef'.repeat(8),
  created_at: 1775867000,
  kind: 'intent.broadcast',
  payload: { type: 'experience', data: {} },
  tags: [],
  visibility: 'public',
  operator_pubkey: 'deadbeef'.repeat(8),
  sig: 'abc'.repeat(21) + 'ab'
}
expect(event.v).toBe(1)

// Test 2: Protocol-layer kinds only — no experience.xxx allowed
const kind: IntentKind = 'intent.broadcast'
// TypeScript compile error expected if: 'experience.publish' assigned to
IntentKind

// Test 3: ExperiencePayload is application-layer, extends IntentPayload
const payload: ExperiencePayload = {
  type: 'experience',
  data: {
    what: 'Docker DNS fix',
    tried: 'modified /etc/resolv.conf',
    outcome: 'succeeded',
    learned: 'restart container to clear DNS cache',
  }
}
expect(payload.type).toBe('experience')
```

```
// Test 4: OperatorKey and AgentKey are distinct types
const opKey: OperatorKey = { publicKey: 'a'.repeat(64), privateKey: new
Uint8Array(32) }
const agentKey: AgentKey = {
  publicKey: 'b'.repeat(64),
  privateKey: new Uint8Array(32),
  delegatedBy: 'a'.repeat(64),
  expiresAt: Math.floor(Date.now() / 1000) + 90 * 86400
}
expect(agentKey.delegatedBy).toBe(opKey.publicKey)

// Test 5: ExperienceScope is optional, structured
const scope: ExperienceScope = {
  versions: ['docker>=24', 'bun>=1.0'],
  platforms: ['linux', 'macos'],
  context: 'production'
}

// Test 6: tsc --noEmit passes with zero errors
```

Implement:

```
// SerendipEvent — minimal envelope
export interface SerendipEvent {
  v: 1                          // protocol version, immutable
  id: string                    // SHA-256 of canonical content (hex)
  pubkey: string                // publisher Agent public key (hex)
  created_at: number            // unix timestamp
  kind: SerendipKind
  payload: IntentPayload
  tags: string[]
  visibility: 'public' | 'private'
  operator_pubkey: string       // Operator master key public key (hex)
  sig: string                   // Ed25519 signature (hex)
}

// Protocol-layer kinds ONLY — never application kinds here
export type IntentKind =
  | 'intent.broadcast'
  | 'intent.match'
  | 'intent.verify'
  | 'intent.subscribe'

export type IdentityKind =
  | 'identity.register'
```

```
  | 'identity.delegate'
  | 'identity.revoke'

export type SerendipKind = IntentKind | IdentityKind

// Generic intent payload — protocol doesn't care what's inside
export interface IntentPayload {
  type: string      // application-defined: 'experience' | 'capability' |
etc.
  data: unknown
}

// AgentXP application-layer: experience specialization
export interface ExperienceData {
  what: string
  tried: string
  outcome: 'succeeded' | 'failed' | 'partial' | 'inconclusive'
  learned: string
  scope?: ExperienceScope
}

export interface ExperienceScope {
  versions?: string[]
  platforms?: string[]
  context?: string
}

export interface ExperiencePayload extends IntentPayload {
  type: 'experience'
  data: ExperienceData
}

// Identity key types
export interface OperatorKey {
  publicKey: string       // hex
  privateKey: Uint8Array
}

export interface AgentKey {
  publicKey: string       // hex
  privateKey: Uint8Array
  delegatedBy: string     // operator public key
  expiresAt: number       // unix timestamp
  agentId?: string        // human-readable name
}
```

Verify: `tsc --noEmit` passes. All tests green. Commit: `feat(protocol): A1 type definitions`

A2: Ed25519 Key Generation

Goal: Generate Operator master key and Agent sub-key. Delegation is cryptographically verifiable.

Directory: `packages/protocol/src/keys.ts`

Tests to write first (`packages/protocol/tests/keys.test.ts`):

```
// Test 1: generateOperatorKey produces valid key pair
const key = await generateOperatorKey()
expect(key.publicKey).toHaveLength(64)      // 32 bytes hex
expect(key.privateKey).toHaveLength(32)     // Uint8Array

// Test 2: delegateAgentKey produces verifiable delegation
const opKey = await generateOperatorKey()
const agentKey = await delegateAgentKey(opKey, 'my-agent', 90)
expect(agentKey.delegatedBy).toBe(opKey.publicKey)
expect(agentKey.expiresAt).toBeGreaterThan(Math.floor(Date.now() / 1000))
expect(agentKey.expiresAt).toBeLessThan(Math.floor(Date.now() / 1000) + 91
* 86400)

// Test 3: Solo developer mode — operator key IS agent key
const soloKey = await generateOperatorKey()
const selfDelegate = await delegateAgentKey(soloKey, 'solo-agent', 365)
expect(selfDelegate.delegatedBy).toBe(soloKey.publicKey)

// Test 4: revokeAgentKey produces a signed revocation event
const revoke = await revokeAgentKey(opKey, agentKey.publicKey)
expect(revoke.kind).toBe('identity.revoke')
expect(revoke.pubkey).toBe(opKey.publicKey)
expect(revoke.payload.data.revokedKey).toBe(agentKey.publicKey)

// Test 5: different operator keys are unique
const key1 = await generateOperatorKey()
const key2 = await generateOperatorKey()
expect(key1.publicKey).not.toBe(key2.publicKey)
```

Implement:

```
import { ed25519 } from '@noble/curves/ed25519'
import { randomBytes } from '@noble/hashes/utils'

export async function generateOperatorKey(): Promise<OperatorKey> {
```

```
  const privateKey = randomBytes(32)
  const publicKey = bytesToHex(ed25519.getPublicKey(privateKey))
  return { publicKey, privateKey }
}

export async function delegateAgentKey(
  operatorKey: OperatorKey,
  agentId: string,
  ttlDays: number
): Promise<AgentKey> {
  const privateKey = randomBytes(32)
  const publicKey = bytesToHex(ed25519.getPublicKey(privateKey))
  const expiresAt = Math.floor(Date.now() / 1000) + ttlDays * 86400
  return {
    publicKey,
    privateKey,
    delegatedBy: operatorKey.publicKey,
    expiresAt,
    agentId
  }
}

export async function revokeAgentKey(
  operatorKey: OperatorKey,
  agentPubkey: string
): Promise<SerendipEvent> {
  // returns a signed identity.revoke event
  // implementation uses signEvent from events.ts
}
```

Verify: All 5 tests green. Commit: `feat(protocol): A2 Ed25519 key generation`

A3: Event Signing & Verification

Goal: Sign an event. Verify a signature. Detect tampering.

Directory: `packages/protocol/src/events.ts`

Tests to write first (`packages/protocol/tests/events.test.ts`):

```
// Test 1: createEvent builds canonical unsigned event
const event = createEvent('intent.broadcast', payload, ['docker', 'dns'])
expect(event.kind).toBe('intent.broadcast')
expect(event.v).toBe(1)
expect(event.sig).toBeUndefined()   // not signed yet

// Test 2: signEvent adds valid signature and id
```

```
const signed = await signEvent(event, agentKey)
expect(signed.sig).toHaveLength(128)   // 64 bytes hex
expect(signed.id).toHaveLength(64)     // 32 bytes hex

// Test 3: verifyEvent returns true for valid signature
expect(await verifyEvent(signed)).toBe(true)

// Test 4: tampered content fails verification
const tampered = { ...signed, payload: { ...signed.payload, data: { what:
'tampered' } } }
expect(await verifyEvent(tampered)).toBe(false)

// Test 5: tampered id fails verification
const badId = { ...signed, id: 'a'.repeat(64) }
expect(await verifyEvent(badId)).toBe(false)

// Test 6: canonicalize is deterministic (same input = same output)
const c1 = canonicalize(event)
const c2 = canonicalize({ ...event })   // different object, same content
expect(c1).toBe(c2)

// Test 7: expired agent key still verifies (revocation checked at relay,
not here)
const expiredKey = { ...agentKey, expiresAt: Math.floor(Date.now() / 1000)
- 1 }
const signedExpired = await signEvent(event, expiredKey)
expect(await verifyEvent(signedExpired)).toBe(true)  // expiry = relay
concern
```

Implement:

```
export function createEvent(
  kind: SerendipKind,
  payload: IntentPayload,
  tags: string[]
): Omit<SerendipEvent, 'sig'> {
  // builds event without signature
  // id not yet set (needs signing)
}

export async function signEvent(
  event: Omit<SerendipEvent, 'sig' | 'id'>,
  agentKey: AgentKey
): Promise<SerendipEvent> {
  const canonical = canonicalize(event)
  const id = sha256hex(canonical)
```

```
  const sig = bytesToHex(ed25519.sign(hexToBytes(id), agentKey.privateKey))
  return { ...event, id, sig, pubkey: agentKey.publicKey, operator_pubkey:
agentKey.delegatedBy }
}

export async function verifyEvent(event: SerendipEvent): Promise<boolean> {
  // 1. recompute id from canonical content (excluding id and sig fields)
  // 2. verify id matches event.id
  // 3. verify sig against id using event.pubkey
  // returns false on any mismatch, never throws
}

export function canonicalize(event: Partial<SerendipEvent>): string {
  // deterministic JSON: sorted keys, no whitespace, no sig/id fields
}
```

Verify: All 7 tests green. Commit: `feat(protocol): A3 event signing and verification`

A4: Merkle Hash

Goal: Build Merkle root over a set of events. Prove an event is included. Verify the proof.

Directory: `packages/protocol/src/merkle.ts`

Tests to write first (`packages/protocol/tests/merkle.test.ts`):

```
// Test 1: buildMerkleRoot returns consistent hash for same events
const events = [event1, event2, event3]
const root1 = buildMerkleRoot(events)
const root2 = buildMerkleRoot([...events])
expect(root1).toBe(root2)
expect(root1).toHaveLength(64)  // SHA-256 hex

// Test 2: different events produce different root
const root3 = buildMerkleRoot([event1, event2])
expect(root1).not.toBe(root3)

// Test 3: getMerkleProof returns valid proof for included event
const proof = getMerkleProof(events, event1.id)
expect(proof).not.toBeNull()
expect(Array.isArray(proof)).toBe(true)

// Test 4: verifyMerkleProof confirms included event
expect(verifyMerkleProof(event1.id, proof!, root1)).toBe(true)

// Test 5: verifyMerkleProof rejects non-included event
expect(verifyMerkleProof('fake-id-' + 'a'.repeat(56), proof!,
```

```
root1)).toBe(false)

// Test 6: single event tree works
const singleRoot = buildMerkleRoot([event1])
const singleProof = getMerkleProof([event1], event1.id)
expect(verifyMerkleProof(event1.id, singleProof!, singleRoot)).toBe(true)

// Test 7: getMerkleProof returns null for non-included event
expect(getMerkleProof(events, 'not-in-tree')).toBeNull()
```

Implement:

```
export function buildMerkleRoot(events: SerendipEvent[]): string {
  // standard binary Merkle tree over event ids
  // leaf = sha256(event.id)
  // node = sha256(left + right)
  // odd number of leaves: duplicate last leaf
}

export function getMerkleProof(
  events: SerendipEvent[],
  eventId: string
): string[] | null {
  // returns array of sibling hashes needed to reconstruct root
  // returns null if eventId not in events
}

export function verifyMerkleProof(
  eventId: string,
  proof: string[],
  root: string
): boolean {
  // reconstruct root from eventId + proof
  // compare to provided root
}
```

Verify: All 7 tests green. Run integration test: A2 key → A3 sign → A4 Merkle all connected.
Commit: `feat(protocol): A4 Merkle hash integrity`

Phase A Integration Test

After A4 complete, run full chain:

```
// packages/protocol/tests/integration.test.ts

// 1. Generate keys
```

```
const opKey = await generateOperatorKey()
const agentKey = await delegateAgentKey(opKey, 'test-agent', 90)

// 2. Create and sign event
const payload: ExperiencePayload = {
  type: 'experience',
  data: { what: 'test', tried: 'test', outcome: 'succeeded', learned:
'test' }
}
const event = createEvent('intent.broadcast', payload, ['test'])
const signed = await signEvent(event, agentKey)

// 3. Verify signature
expect(await verifyEvent(signed)).toBe(true)

// 4. Build Merkle tree and verify inclusion
const root = buildMerkleRoot([signed])
const proof = getMerkleProof([signed], signed.id)
expect(verifyMerkleProof(signed.id, proof!, root)).toBe(true)

// 5. Revoke key and check revocation event is signed by operator
const revoke = await revokeAgentKey(opKey, agentKey.publicKey)
expect(await verifyEvent(revoke)).toBe(true)
expect(revoke.kind).toBe('identity.revoke')
```

All 5 steps pass → Phase A complete → Commit: `feat(protocol): Phase A integration test passing`

Phase A spec complete. Phase B spec next.